

Medicine Management

Create, Read, Update and Delete (CRUD) of medicine records

Name **Jules Aron P. Timbas**
Section **INF241**
Database **dbPharma**
Table **tblMedicine**

CONTENTS

- | | | |
|---|---|---------------|
| 1 | Database – dbPharma.sql and tblMedicine | Figures 1–2 |
| 2 | PHP API – db.php and index.php (htdocs/api) | Figures 3–7 |
| 3 | React JS – App.jsx | Figures 8–14 |
| 4 | Output – Create, Read, Update and Delete in the browser | Figures 15–22 |
| 5 | Database after the CRUD operations | Figure 23 |

TBLMEDICINE STRUCTURE

Column	Data type	Constraints
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
brand_name	VARCHAR(100)	NOT NULL
generic_name	VARCHAR(100)	NOT NULL
date_manufactured	DATE	NOT NULL
date_expired	DATE	NOT NULL, CHECK (date_expired > date_manufactured)
manufacturer	VARCHAR(150)	NOT NULL
batch_number	VARCHAR(50)	NOT NULL, UNIQUE

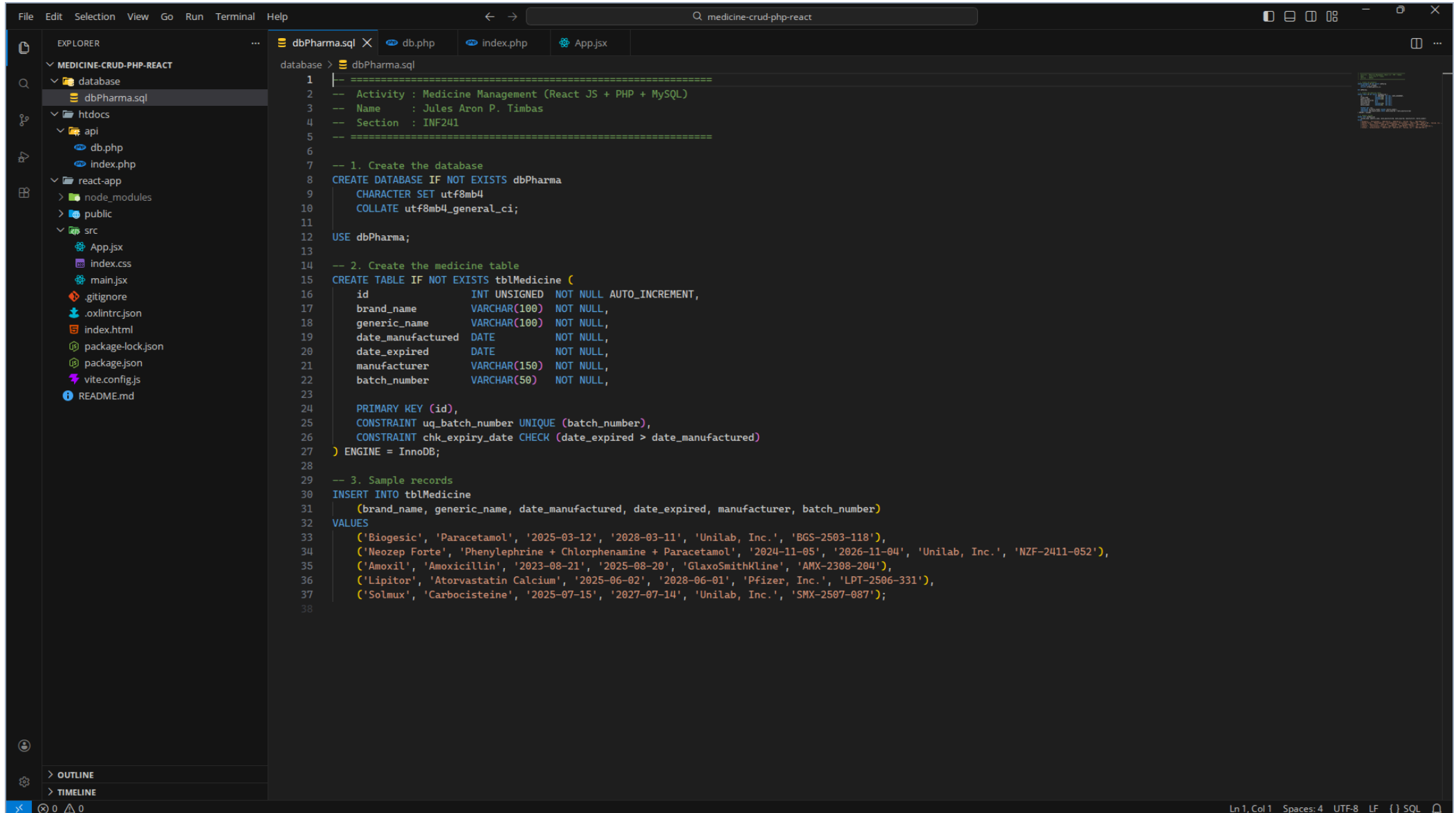
Screenshots are full screen (1920 × 1080). The code is shown in the editor; the output is the React app running at localhost:5173 with the PHP API at localhost/api/index.php.

Figure 1. database/dbPharma.sql

Jules Aron P. Timbas

INF241

Creates the dbPharma database, the tblMedicine table (data types and constraints) and 5 sample records.



```
1  |-----
2  -- Activity : Medicine Management (React JS + PHP + MySQL)
3  -- Name      : Jules Aron P. Timbas
4  -- Section   : INF241
5  |-----
6
7  -- 1. Create the database
8  CREATE DATABASE IF NOT EXISTS dbPharma
9      CHARACTER SET utf8mb4
10     COLLATE utf8mb4_general_ci;
11
12  USE dbPharma;
13
14  -- 2. Create the medicine table
15  CREATE TABLE IF NOT EXISTS tblMedicine (
16      id                INT UNSIGNED NOT NULL AUTO_INCREMENT,
17      brand_name         VARCHAR(100) NOT NULL,
18      generic_name       VARCHAR(100) NOT NULL,
19      date_manufactured  DATE          NOT NULL,
20      date_expired       DATE          NOT NULL,
21      manufacturer       VARCHAR(150) NOT NULL,
22      batch_number       VARCHAR(50)  NOT NULL,
23
24      PRIMARY KEY (id),
25      CONSTRAINT uq_batch_number UNIQUE (batch_number),
26      CONSTRAINT chk_expiry_date CHECK (date_expired > date_manufactured)
27  ) ENGINE = InnoDB;
28
29  -- 3. Sample records
30  INSERT INTO tblMedicine
31      (brand_name, generic_name, date_manufactured, date_expired, manufacturer, batch_number)
32  VALUES
33      ('Biogesic', 'Paracetamol', '2025-03-12', '2028-03-11', 'Unilab, Inc.', 'BGS-2503-118'),
34      ('Neozep Forte', 'Phenylephrine + Chlorphenamine + Paracetamol', '2024-11-05', '2026-11-04', 'Unilab, Inc.', 'NZF-2411-052'),
35      ('Amoxil', 'Amoxicillin', '2023-08-21', '2025-08-20', 'GlaxoSmithKline', 'AMX-2308-204'),
36      ('Lipitor', 'Atorvastatin Calcium', '2025-06-02', '2028-06-01', 'Pfizer, Inc.', 'LPT-2506-331'),
37      ('Solmux', 'Carbocisteine', '2025-07-15', '2027-07-14', 'Unilab, Inc.', 'SMX-2507-087');
38
```

Figure 2. tblMedicine in MySQL (MariaDB)

Jules Aron P. Timbas

INF241

DESCRIBE shows the data types; the PRIMARY KEY, UNIQUE and CHECK constraints are listed with the sample records.

The screenshot shows a code editor with the following content:

```

14 -- 2. Create the medicine table
15 CREATE TABLE IF NOT EXISTS tblMedicine (
16     id            INT UNSIGNED NOT NULL AUTO_INCREMENT,
17     brand_name    VARCHAR(100) NOT NULL,
18     generic_name   VARCHAR(100) NOT NULL,
19     date_manufactured DATE NOT NULL,
20     date_expired   DATE NOT NULL,
21     manufacturer   VARCHAR(150) NOT NULL,
22     batch_number   VARCHAR(50) NOT NULL,
23
24     PRIMARY KEY (id),
25     CONSTRAINT uq_batch_number UNIQUE (batch_number),
26     CONSTRAINT chk_expiry_date CHECK (date_expired > date_manufactured)
27 ) ENGINE = InnoDB;
28
29 -- 3. Sample records

```

The terminal output shows the results of the following queries:

```

MariaDB [dbPharma]> DESCRIBE tblMedicine;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int(10) unsigned | NO | PRI | NULL | auto_increment |
| brand_name | varchar(100) | NO | | NULL | |
| generic_name | varchar(100) | NO | | NULL | |
| date_manufactured | date | NO | | NULL | |
| date_expired | date | NO | | NULL | |
| manufacturer | varchar(150) | NO | | NULL | |
| batch_number | varchar(50) | NO | UNI | NULL | |
+-----+-----+-----+-----+-----+-----+
7 rows in set (0.001 sec)

MariaDB [dbPharma]> SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE FROM information_schema.TABLE_CONSTRAINTS WHERE TABLE_SCHEMA = 'dbPharma' AND TABLE_NAME = 'tblMedicine';
+-----+-----+
| CONSTRAINT_NAME | CONSTRAINT_TYPE |
+-----+-----+
| PRIMARY | PRIMARY KEY |
| uq_batch_number | UNIQUE |
| chk_expiry_date | CHECK |
+-----+-----+
3 rows in set (0.000 sec)

MariaDB [dbPharma]> SELECT * FROM tblMedicine;
+----+-----+-----+-----+-----+-----+-----+
| id | brand_name | generic_name | date_manufactured | date_expired | manufacturer | batch_number |
+----+-----+-----+-----+-----+-----+-----+
| 1 | Biogesic | Paracetamol | 2025-03-12 | 2028-03-11 | Unilab, Inc. | BGS-2503-118 |
| 2 | Neozepe Forte | Phenylephrine + Chlorphenamine + Paracetamol | 2024-11-05 | 2026-11-04 | Unilab, Inc. | NZF-2411-052 |
| 3 | Amoxil | Amoxicillin | 2023-08-21 | 2025-08-20 | GlaxoSmithKline | AMX-2308-204 |
| 4 | Lipitor | Atorvastatin Calcium | 2025-06-02 | 2028-06-01 | Pfizer, Inc. | LPT-2506-331 |
| 5 | Solmux | Carbocisteine | 2025-07-15 | 2027-07-14 | Unilab, Inc. | SMX-2507-087 |
+----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.001 sec)

MariaDB [dbPharma]>

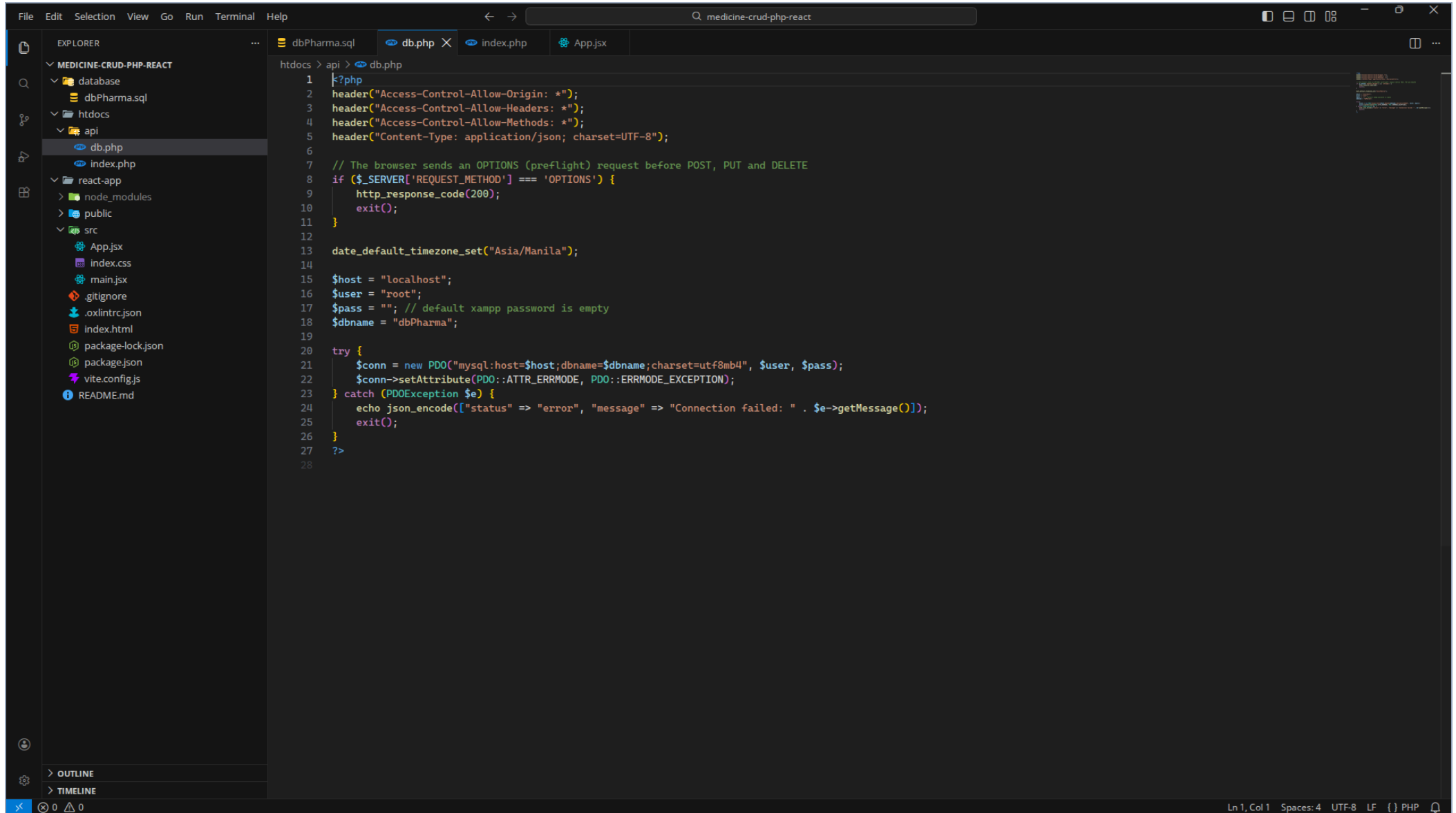
```

Figure 3. htdocs/api/db.php

CORS / JSON headers and the PDO connection to the dbPharma database.

Jules Aron P. Timbas

INF241



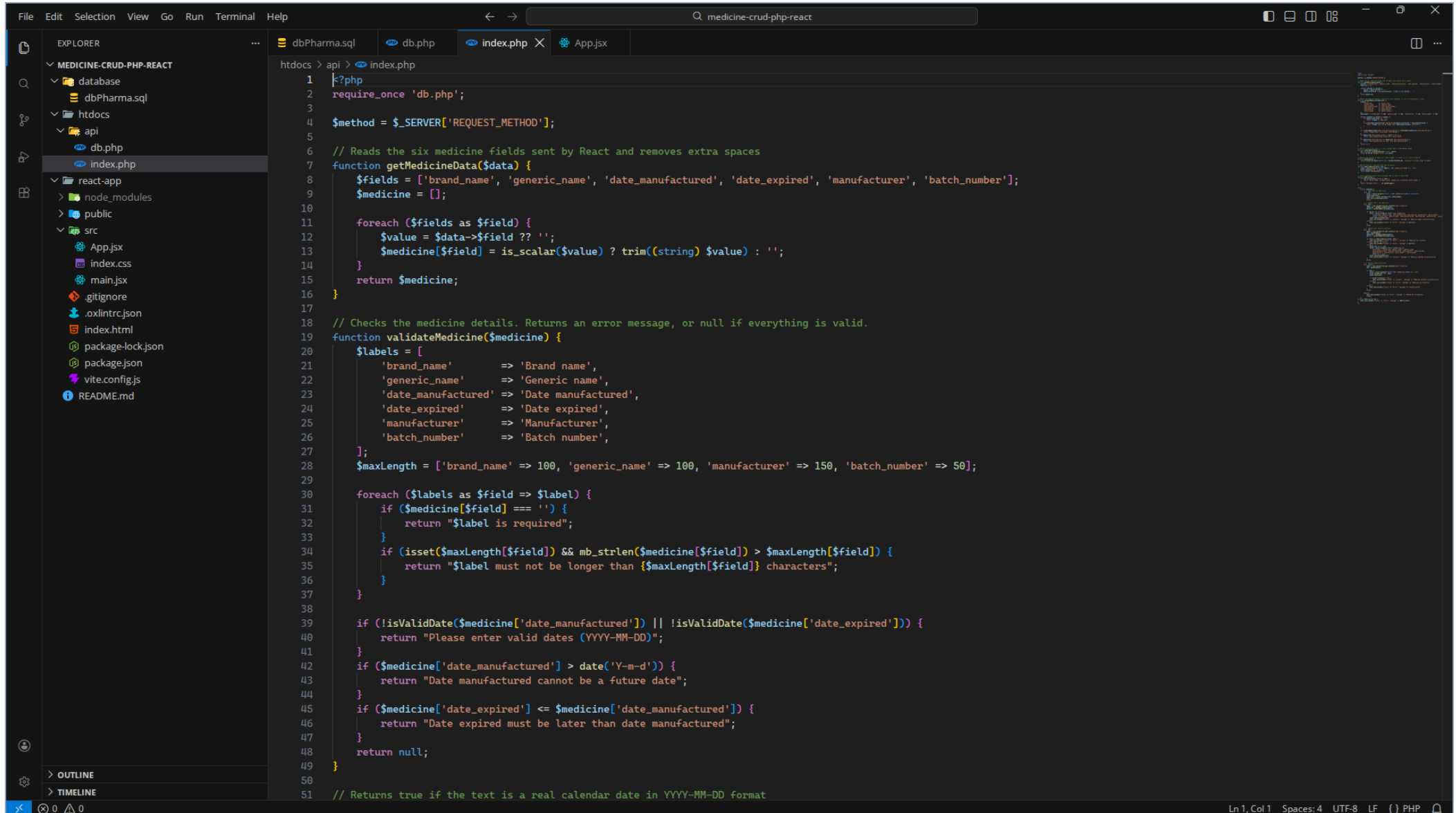
```
1 <?php
2 header("Access-Control-Allow-Origin: *");
3 header("Access-Control-Allow-Headers: *");
4 header("Access-Control-Allow-Methods: *");
5 header("Content-Type: application/json; charset=UTF-8");
6
7 // The browser sends an OPTIONS (preflight) request before POST, PUT and DELETE
8 if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
9     http_response_code(200);
10    exit();
11 }
12
13 date_default_timezone_set("Asia/Manila");
14
15 $host = "localhost";
16 $user = "root";
17 $pass = ""; // default xampp password is empty
18 $dbname = "dbPharma";
19
20 try {
21     $conn = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $user, $pass);
22     $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
23 } catch (PDOException $e) {
24     echo json_encode(["status" => "error", "message" => "Connection failed: " . $e->getMessage()]);
25     exit();
26 }
27 ?>
```

Figure 4. htdocs/api/index.php (1 of 4) – lines 1–50

Reads the medicine data sent by React and validates every field.

Jules Aron P. Timbas

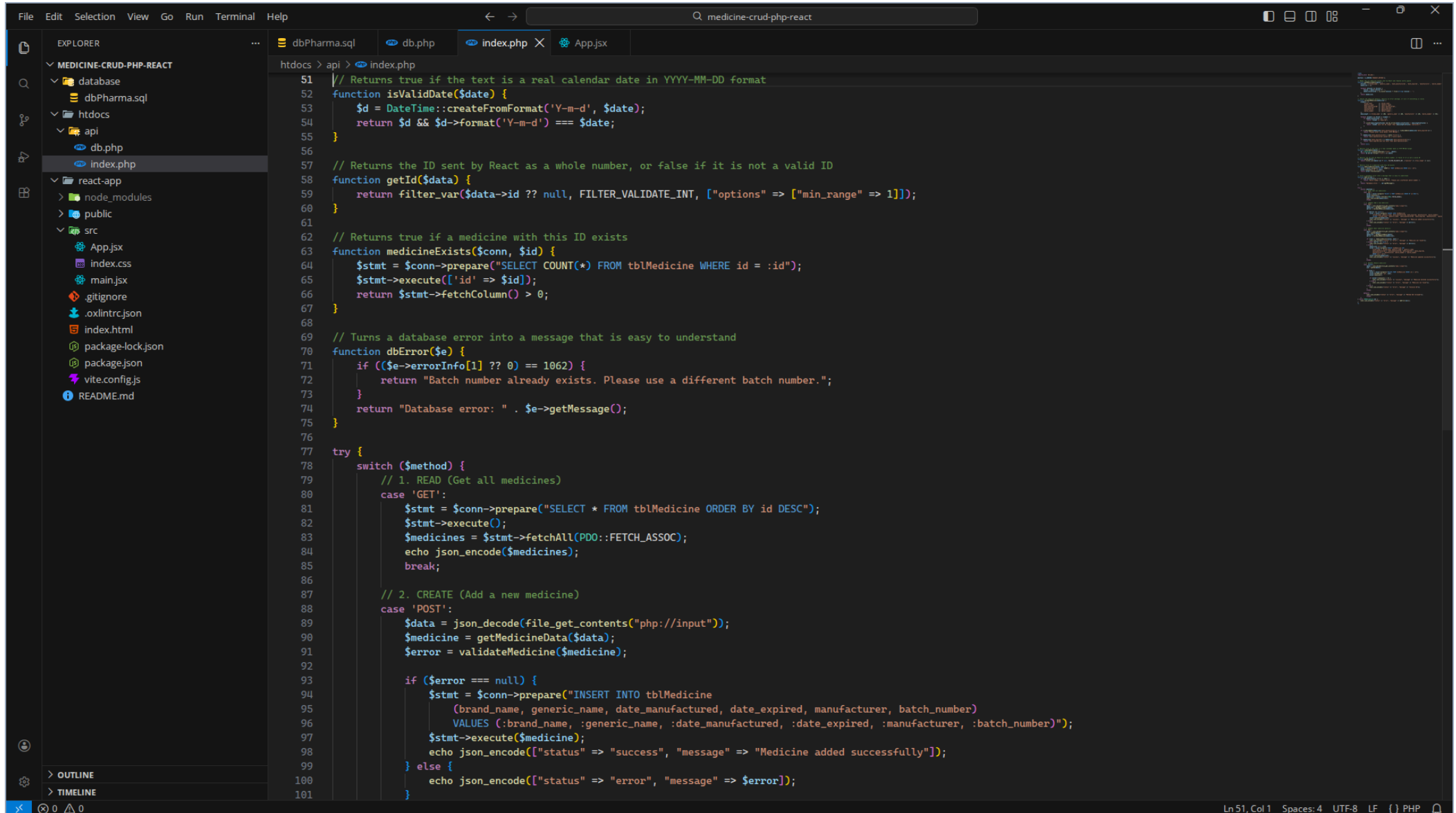
INF241



```
1 1?php
2 require_once 'db.php';
3
4 $method = $_SERVER['REQUEST_METHOD'];
5
6 // Reads the six medicine fields sent by React and removes extra spaces
7 function getMedicineData($data) {
8     $fields = ['brand_name', 'generic_name', 'date_manufactured', 'date_expired', 'manufacturer', 'batch_number'];
9     $medicine = [];
10
11     foreach ($fields as $field) {
12         $value = $data->$field ?? '';
13         $medicine[$field] = is_scalar($value) ? trim((string) $value) : '';
14     }
15     return $medicine;
16 }
17
18 // Checks the medicine details. Returns an error message, or null if everything is valid.
19 function validateMedicine($medicine) {
20     $labels = [
21         'brand_name' => 'Brand name',
22         'generic_name' => 'Generic name',
23         'date_manufactured' => 'Date manufactured',
24         'date_expired' => 'Date expired',
25         'manufacturer' => 'Manufacturer',
26         'batch_number' => 'Batch number',
27     ];
28     $maxLength = ['brand_name' => 100, 'generic_name' => 100, 'manufacturer' => 150, 'batch_number' => 50];
29
30     foreach ($labels as $field => $label) {
31         if ($medicine[$field] === '') {
32             return "$label is required";
33         }
34         if (isset($maxLength[$field]) && mb_strlen($medicine[$field]) > $maxLength[$field]) {
35             return "$label must not be longer than {$maxLength[$field]} characters";
36         }
37     }
38
39     if (!isValidDate($medicine['date_manufactured']) || !isValidDate($medicine['date_expired'])) {
40         return "Please enter valid dates (YYYY-MM-DD)";
41     }
42     if ($medicine['date_manufactured'] > date('Y-m-d')) {
43         return "Date manufactured cannot be a future date";
44     }
45     if ($medicine['date_expired'] <= $medicine['date_manufactured']) {
46         return "Date expired must be later than date manufactured";
47     }
48     return null;
49 }
50
51 // Returns true if the text is a real calendar date in YYYY-MM-DD format
```

Figure 5. htdocs/api/index.php (2 of 4) – lines 51–100

Helper functions, READ (GET) and CREATE (POST).



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows the project structure for 'MEDICINE-CRUD-PHP-REACT', including 'database', 'htdocs', 'react-app', and 'src'. The code editor displays the contents of 'htdocs > api > index.php' from line 51 to 101. The code includes helper functions for date validation, ID retrieval, and database error handling, followed by a switch statement for GET and POST requests.

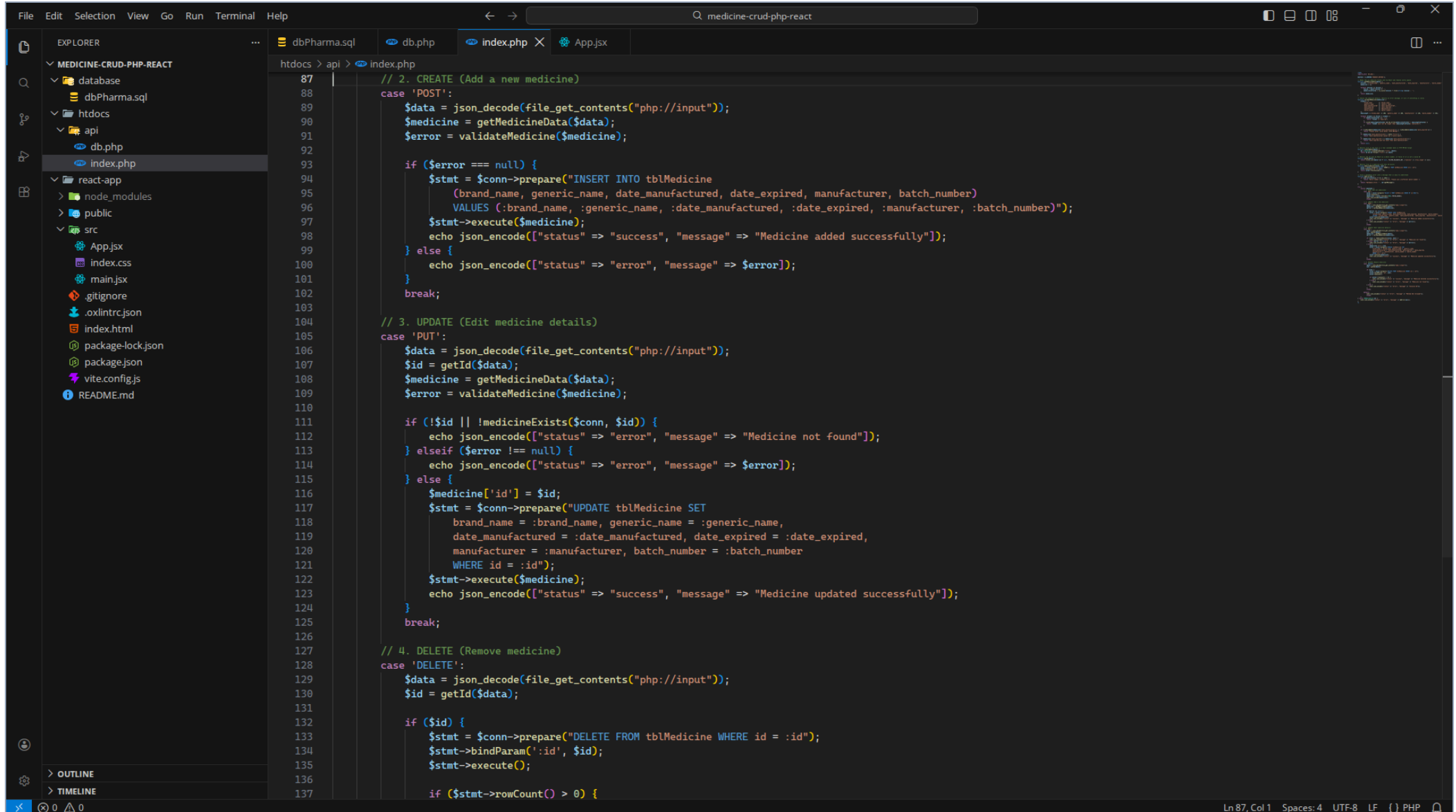
```
51 // Returns true if the text is a real calendar date in YYYY-MM-DD format
52 function isValidDate($date) {
53     $d = DateTime::createFromFormat('Y-m-d', $date);
54     return $d && $d->format('Y-m-d') === $date;
55 }
56
57 // Returns the ID sent by React as a whole number, or false if it is not a valid ID
58 function getId($data) {
59     return filter_var($data->id ?? null, FILTER_VALIDATE_INT, ["options" => ["min_range" => 1]]);
60 }
61
62 // Returns true if a medicine with this ID exists
63 function medicineExists($conn, $id) {
64     $stmt = $conn->prepare("SELECT COUNT(*) FROM tblMedicine WHERE id = :id");
65     $stmt->execute(['id' => $id]);
66     return $stmt->fetchColumn() > 0;
67 }
68
69 // Turns a database error into a message that is easy to understand
70 function dbError($e) {
71     if (($e->errorInfo[1] ?? 0) == 1062) {
72         return "Batch number already exists. Please use a different batch number.";
73     }
74     return "Database error: " . $e->getMessage();
75 }
76
77 try {
78     switch ($method) {
79         // 1. READ (Get all medicines)
80         case 'GET':
81             $stmt = $conn->prepare("SELECT * FROM tblMedicine ORDER BY id DESC");
82             $stmt->execute();
83             $medicines = $stmt->fetchAll(PDO::FETCH_ASSOC);
84             echo json_encode($medicines);
85             break;
86
87         // 2. CREATE (Add a new medicine)
88         case 'POST':
89             $data = json_decode(file_get_contents("php://input"));
90             $medicine = getMedicineData($data);
91             $error = validateMedicine($medicine);
92
93             if ($error === null) {
94                 $stmt = $conn->prepare("INSERT INTO tblMedicine
95                     (brand_name, generic_name, date_manufactured, date_expired, manufacturer, batch_number)
96                     VALUES (:brand_name, :generic_name, :date_manufactured, :date_expired, :manufacturer, :batch_number)");
97                 $stmt->execute($medicine);
98                 echo json_encode(["status" => "success", "message" => "Medicine added successfully"]);
99             } else {
100                 echo json_encode(["status" => "error", "message" => $error]);
101             }
102     }
103 }
```

Figure 6. htdocs/api/index.php (3 of 4) – lines 87–136

Jules Aron P. Timbas

INF241

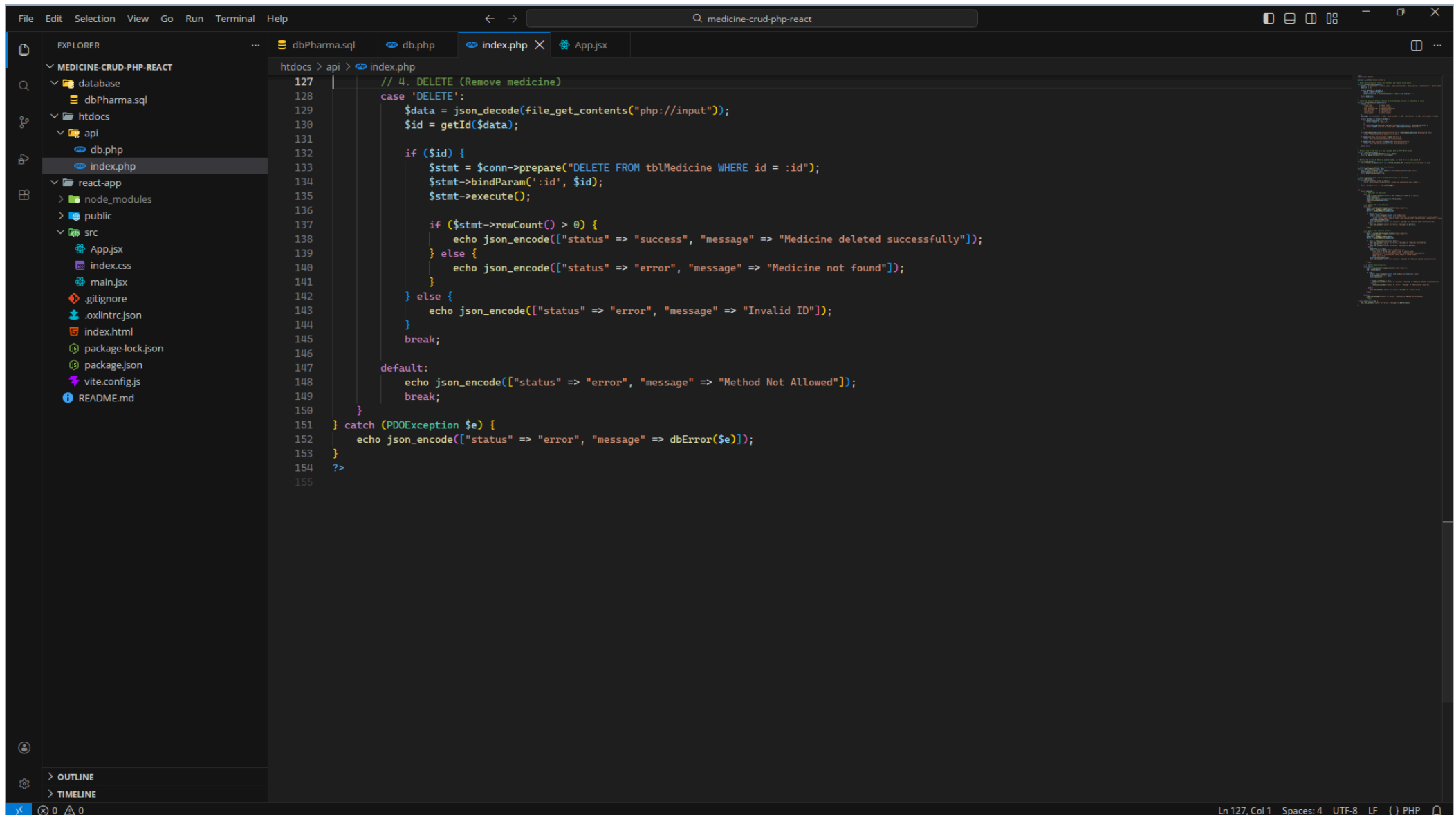
CREATE (POST) and UPDATE (PUT).



```
87 // 2. CREATE (Add a new medicine)
88 case 'POST':
89     $data = json_decode(file_get_contents("php://input"));
90     $medicine = getMedicineData($data);
91     $error = validateMedicine($medicine);
92
93     if ($error === null) {
94         $stmt = $conn->prepare("INSERT INTO tblMedicine
95             (brand_name, generic_name, date_manufactured, date_expired, manufacturer, batch_number)
96             VALUES (:brand_name, :generic_name, :date_manufactured, :date_expired, :manufacturer, :batch_number)");
97         $stmt->execute($medicine);
98         echo json_encode(["status" => "success", "message" => "Medicine added successfully"]);
99     } else {
100         echo json_encode(["status" => "error", "message" => $error]);
101     }
102     break;
103
104 // 3. UPDATE (Edit medicine details)
105 case 'PUT':
106     $data = json_decode(file_get_contents("php://input"));
107     $id = getId($data);
108     $medicine = getMedicineData($data);
109     $error = validateMedicine($medicine);
110
111     if (!$id || !medicineExists($conn, $id)) {
112         echo json_encode(["status" => "error", "message" => "Medicine not found"]);
113     } elseif ($error !== null) {
114         echo json_encode(["status" => "error", "message" => $error]);
115     } else {
116         $medicine['id'] = $id;
117         $stmt = $conn->prepare("UPDATE tblMedicine SET
118             brand_name = :brand_name, generic_name = :generic_name,
119             date_manufactured = :date_manufactured, date_expired = :date_expired,
120             manufacturer = :manufacturer, batch_number = :batch_number
121             WHERE id = :id");
122         $stmt->execute($medicine);
123         echo json_encode(["status" => "success", "message" => "Medicine updated successfully"]);
124     }
125     break;
126
127 // 4. DELETE (Remove medicine)
128 case 'DELETE':
129     $data = json_decode(file_get_contents("php://input"));
130     $id = getId($data);
131
132     if ($id) {
133         $stmt = $conn->prepare("DELETE FROM tblMedicine WHERE id = :id");
134         $stmt->bindParam(':id', $id);
135         $stmt->execute();
136         if ($stmt->rowCount() > 0) {
```

Figure 7. htdocs/api/index.php (4 of 4) – lines 127–154

DELETE and database error handling.



The screenshot shows a code editor with the following structure:

- EXPLORER:** MEDICINE-CRUD-PHP-REACT > database > dbPharma.sql, htdocs > api > db.php, index.php (selected), react-app > node_modules, public, src > App.jsx, index.css, main.jsx, .gitignore, .oxlintrc.json, index.html, package-lock.json, package.json, vite.config.js, README.md.
- FILE:** dbPharma.sql, db.php, index.php (selected), App.jsx.
- EDITOR:** htdocs > api > index.php. The code is as follows:

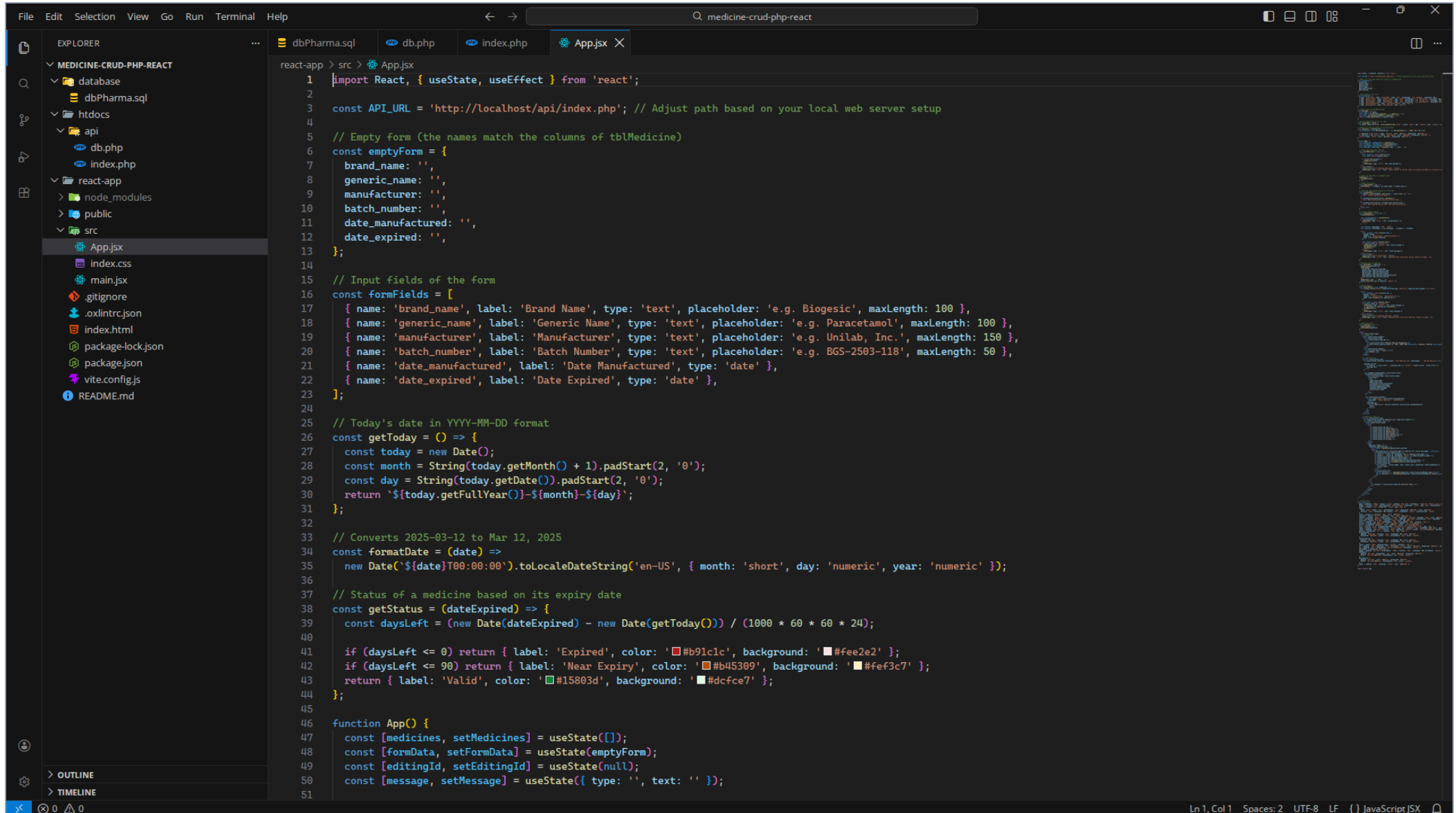
```
127 // 4. DELETE (Remove medicine)
128 case 'DELETE':
129     $data = json_decode(file_get_contents("php://input"));
130     $id = getId($data);
131
132     if ($id) {
133         $stmt = $conn->prepare("DELETE FROM tblMedicine WHERE id = :id");
134         $stmt->bindParam(':id', $id);
135         $stmt->execute();
136
137         if ($stmt->rowCount() > 0) {
138             echo json_encode(["status" => "success", "message" => "Medicine deleted successfully"]);
139         } else {
140             echo json_encode(["status" => "error", "message" => "Medicine not found"]);
141         }
142     } else {
143         echo json_encode(["status" => "error", "message" => "Invalid ID"]);
144     }
145     break;
146
147 default:
148     echo json_encode(["status" => "error", "message" => "Method Not Allowed"]);
149     break;
150 }
151 } catch (PDOException $e) {
152     echo json_encode(["status" => "error", "message" => dbError($e)]);
153 }
154 ?>
155
```
- STATUS BAR:** Ln 127, Col 1 Spaces: 4 UTF-8 LF {} PHP

Figure 8. react-app/src/App.jsx (1 of 7) – lines 1–50

API URL, form fields, date helpers and expiry status.

Jules Aron P. Timbas

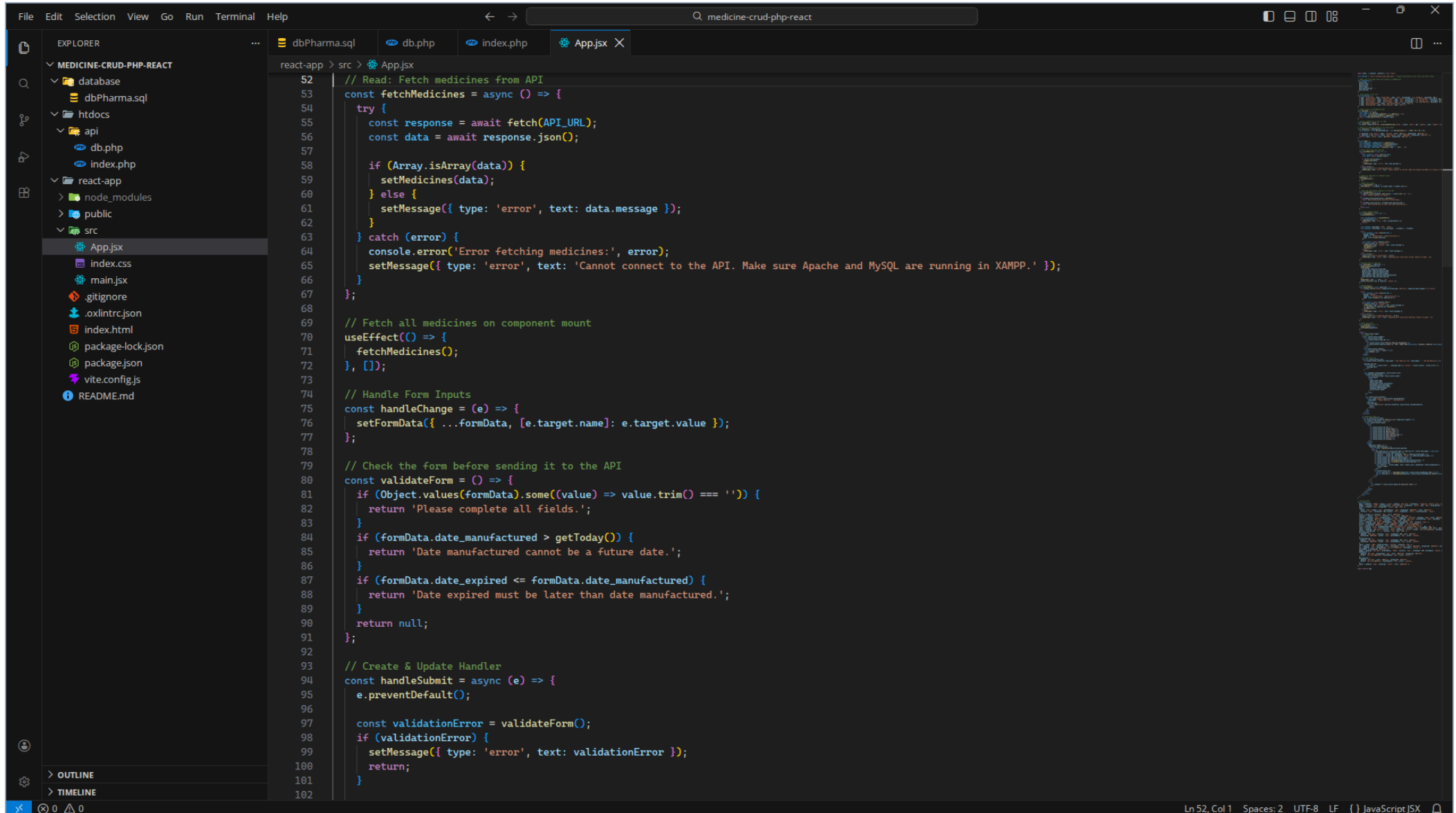
INF241



```
1 import React, { useState, useEffect } from 'react';
2
3 const API_URL = 'http://localhost/api/index.php'; // Adjust path based on your local web server setup
4
5 // Empty form (the names match the columns of tblMedicine)
6 const emptyForm = {
7   brand_name: '',
8   generic_name: '',
9   manufacturer: '',
10  batch_number: '',
11  date_manufactured: '',
12  date_expired: '',
13 };
14
15 // Input fields of the form
16 const formFields = [
17   { name: 'brand_name', label: 'Brand Name', type: 'text', placeholder: 'e.g. Biogesic', maxLength: 100 },
18   { name: 'generic_name', label: 'Generic Name', type: 'text', placeholder: 'e.g. Paracetamol', maxLength: 100 },
19   { name: 'manufacturer', label: 'Manufacturer', type: 'text', placeholder: 'e.g. Unilab, Inc.', maxLength: 150 },
20   { name: 'batch_number', label: 'Batch Number', type: 'text', placeholder: 'e.g. BGS-2503-118', maxLength: 50 },
21   { name: 'date_manufactured', label: 'Date Manufactured', type: 'date' },
22   { name: 'date_expired', label: 'Date Expired', type: 'date' },
23 ];
24
25 // Today's date in YYYY-MM-DD format
26 const getToday = () => {
27   const today = new Date();
28   const month = String(today.getMonth() + 1).padStart(2, '0');
29   const day = String(today.getDate()).padStart(2, '0');
30   return `${today.getFullYear()}-${month}-${day}`;
31 };
32
33 // Converts 2025-03-12 to Mar 12, 2025
34 const formatDate = (date) => {
35   new Date(`${date}T00:00:00`).toLocaleDateString('en-US', { month: 'short', day: 'numeric', year: 'numeric' });
36 };
37
38 // Status of a medicine based on its expiry date
39 const getStatus = (dateExpired) => {
40   const daysLeft = (new Date(dateExpired) - new Date(getToday())) / (1000 * 60 * 60 * 24);
41
42   if (daysLeft <= 0) return { label: 'Expired', color: '#b91c1c', background: '#fee2e2' };
43   if (daysLeft <= 90) return { label: 'Near Expiry', color: '#b45309', background: '#fef3c7' };
44   return { label: 'Valid', color: '#15803d', background: '#d1ecf1' };
45 };
46
47 function App() {
48   const [medicines, setMedicines] = useState([]);
49   const [formData, setFormData] = useState(emptyForm);
50   const [editingId, setEditingId] = useState(null);
51   const [message, setMessage] = useState({ type: '', text: '' });
```

Figure 9. react-app/src/App.jsx (2 of 7) – lines 52–101

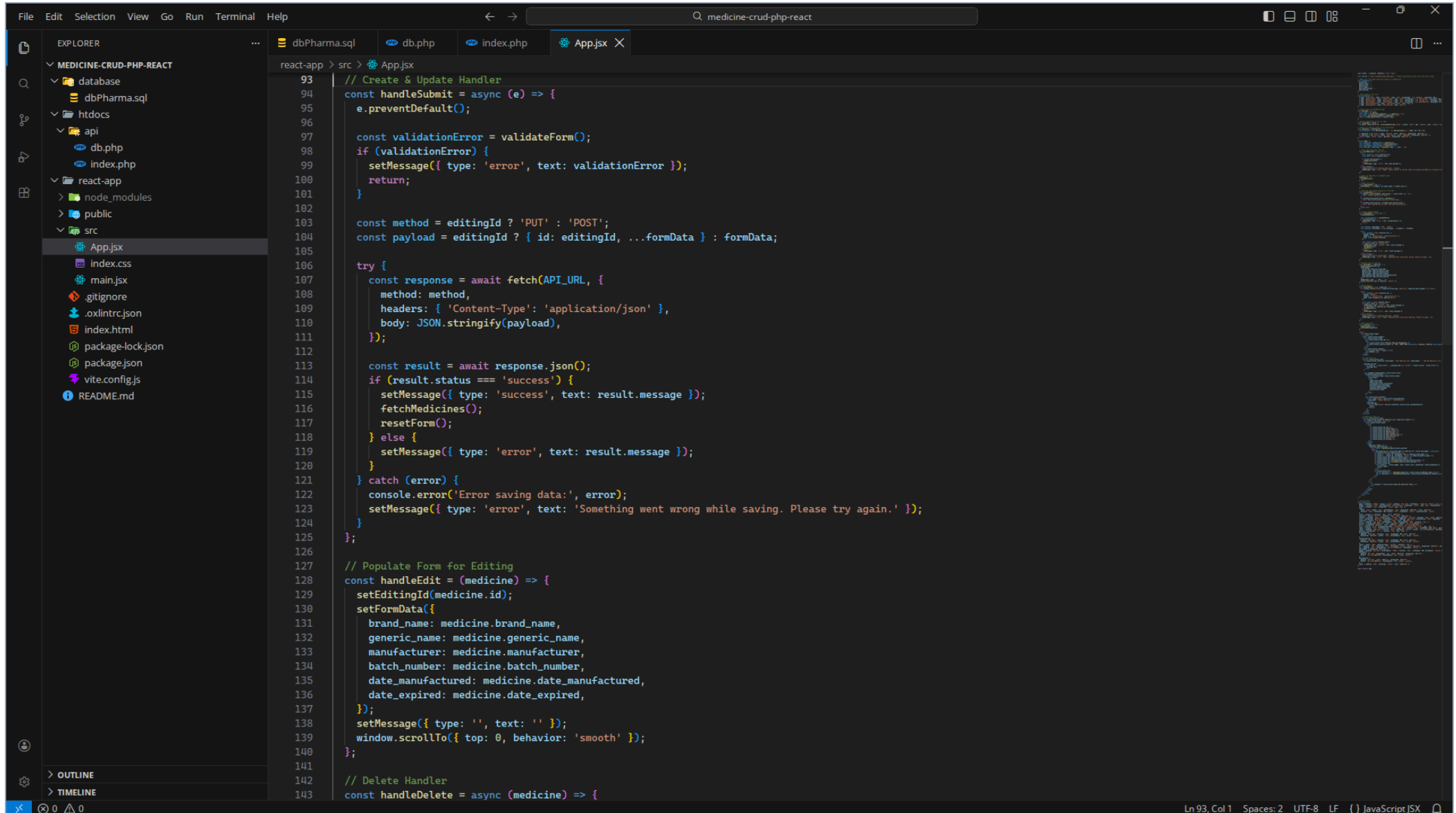
READ: fetch the medicines from the API; form validation.



```
52 // Read: Fetch medicines from API
53 const fetchMedicines = async () => {
54   try {
55     const response = await fetch(API_URL);
56     const data = await response.json();
57
58     if (Array.isArray(data)) {
59       setMedicines(data);
60     } else {
61       setMessage({ type: 'error', text: data.message });
62     }
63   } catch (error) {
64     console.error('Error fetching medicines:', error);
65     setMessage({ type: 'error', text: 'Cannot connect to the API. Make sure Apache and MySQL are running in XAMPP.' });
66   }
67 };
68
69 // Fetch all medicines on component mount
70 useEffect(() => {
71   fetchMedicines();
72 }, []);
73
74 // Handle Form Inputs
75 const handleChange = (e) => {
76   setFormData({ ...formData, [e.target.name]: e.target.value });
77 };
78
79 // Check the form before sending it to the API
80 const validateForm = () => {
81   if (Object.values(formData).some((value) => value.trim() === '')) {
82     return 'Please complete all fields.';
83   }
84   if (formData.date_manufactured > getToday()) {
85     return 'Date manufactured cannot be a future date.';
86   }
87   if (formData.date_expired <= formData.date_manufactured) {
88     return 'Date expired must be later than date manufactured.';
89   }
90   return null;
91 };
92
93 // Create & Update Handler
94 const handleSubmit = async (e) => {
95   e.preventDefault();
96
97   const validationError = validateForm();
98   if (validationError) {
99     setMessage({ type: 'error', text: validationError });
100   }
101   return;
```

Figure 10. react-app/src/App.jsx (3 of 7) – lines 93–142

CREATE & UPDATE: send the form to the API (POST / PUT); load a medicine into the form for editing.



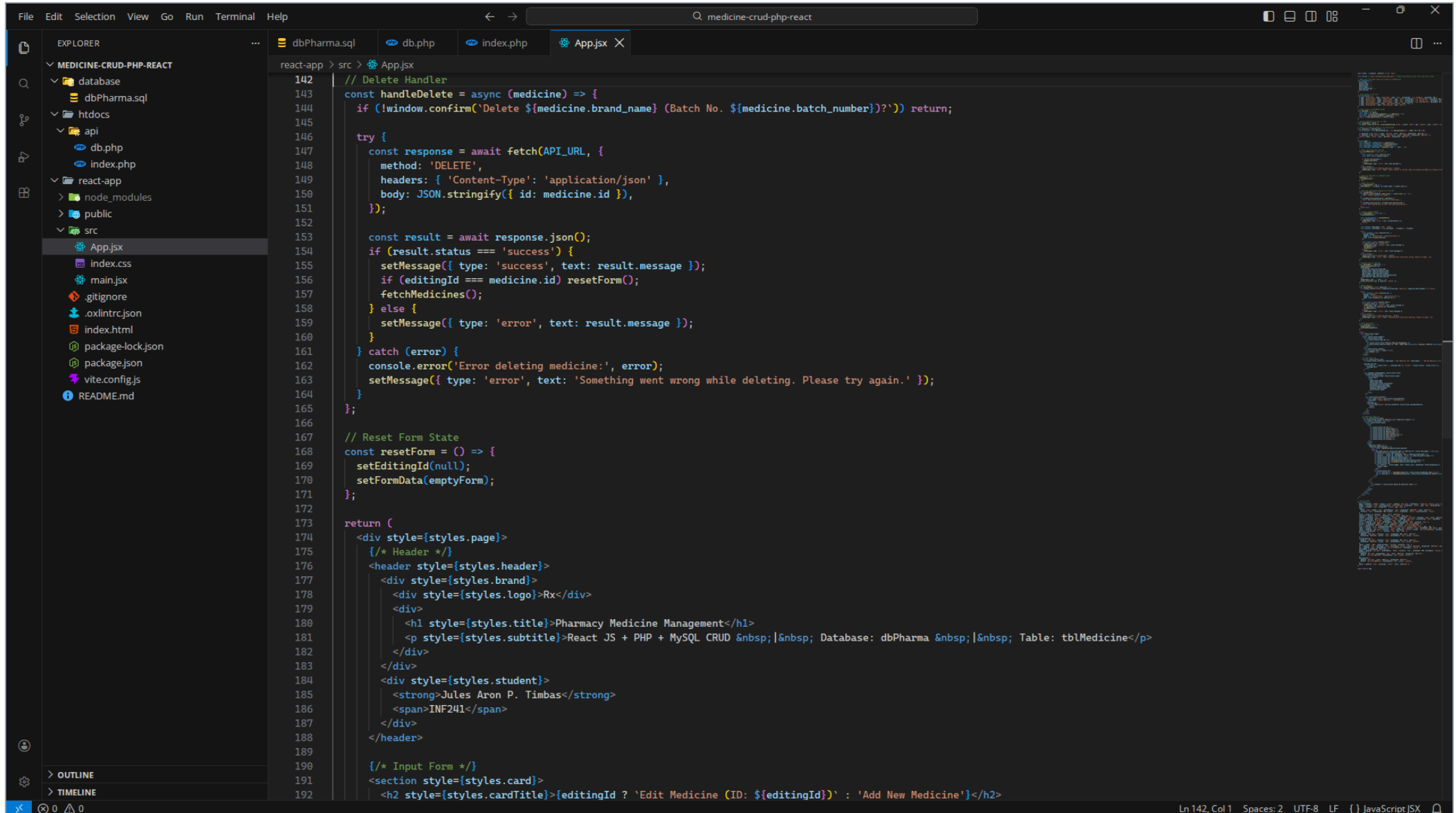
```
93 // Create & Update Handler
94 const handleSubmit = async (e) => {
95   e.preventDefault();
96
97   const validationError = validateForm();
98   if (validationError) {
99     setMessage({ type: 'error', text: validationError });
100    return;
101  }
102
103  const method = editingId ? 'PUT' : 'POST';
104  const payload = editingId ? { id: editingId, ...formData } : formData;
105
106  try {
107    const response = await fetch(API_URL, {
108      method: method,
109      headers: { 'Content-Type': 'application/json' },
110      body: JSON.stringify(payload),
111    });
112
113    const result = await response.json();
114    if (result.status === 'success') {
115      setMessage({ type: 'success', text: result.message });
116      fetchMedicines();
117      resetForm();
118    } else {
119      setMessage({ type: 'error', text: result.message });
120    }
121  } catch (error) {
122    console.error('Error saving data:', error);
123    setMessage({ type: 'error', text: 'Something went wrong while saving. Please try again.' });
124  }
125 };
126
127 // Populate Form for Editing
128 const handleEdit = (medicine) => {
129   setEditingId(medicine.id);
130   setFormData({
131     brand_name: medicine.brand_name,
132     generic_name: medicine.generic_name,
133     manufacturer: medicine.manufacturer,
134     batch_number: medicine.batch_number,
135     date_manufactured: medicine.date_manufactured,
136     date_expired: medicine.date_expired,
137   });
138   setMessage({ type: '', text: '' });
139   window.scrollTo({ top: 0, behavior: 'smooth' });
140 };
141
142 // Delete Handler
143 const handleDelete = async (medicine) => {
```

Figure 11. react-app/src/App.jsx (4 of 7) – lines 142–191

Jules Aron P. Timbas

INF241

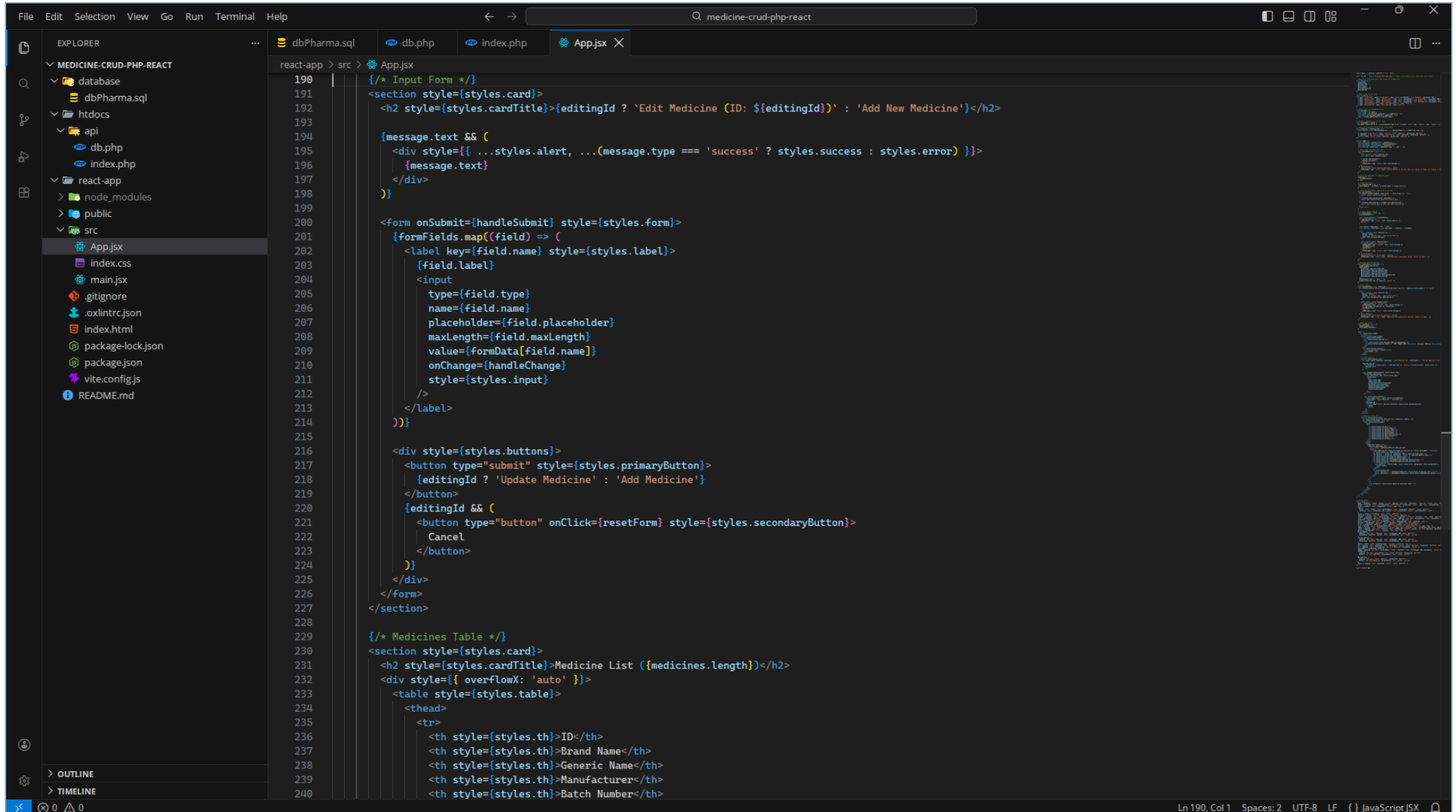
DELETE: remove a medicine (DELETE); reset the form; page header.



```
142 // Delete Handler
143 const handleDelete = async (medicine) => {
144   if (!window.confirm('Delete ${medicine.brand_name} (Batch No. ${medicine.batch_number})?')) return;
145
146   try {
147     const response = await fetch(API_URL, {
148       method: 'DELETE',
149       headers: { 'Content-Type': 'application/json' },
150       body: JSON.stringify({ id: medicine.id }),
151     });
152
153     const result = await response.json();
154     if (result.status === 'success') {
155       setMessage({ type: 'success', text: result.message });
156       if (editingId === medicine.id) resetForm();
157       fetchMedicines();
158     } else {
159       setMessage({ type: 'error', text: result.message });
160     }
161   } catch (error) {
162     console.error('Error deleting medicine:', error);
163     setMessage({ type: 'error', text: 'Something went wrong while deleting. Please try again.' });
164   }
165 };
166
167 // Reset Form State
168 const resetForm = () => {
169   setEditingId(null);
170   setFormData(emptyForm);
171 };
172
173 return (
174   <div style={styles.page}>
175     { /* Header */ }
176     <header style={styles.header}>
177       <div style={styles.brand}>
178         <div style={styles.logo}>Rx</div>
179         <div>
180           <h1 style={styles.title}>Pharmacy Medicine Management</h1>
181           <p style={styles.subtitle}>React JS + PHP + MySQL CRUD &nbsp;&nbsp;&nbsp; Database: dbPharma &nbsp;&nbsp;&nbsp; Table: tblMedicine</p>
182         </div>
183       </div>
184       <div style={styles.student}>
185         <strong>Jules Aron P. Timbas</strong>
186         <span>INF241</span>
187       </div>
188     </header>
189
190     { /* Input Form */ }
191     <section style={styles.card}>
192       <h2 style={styles.cardTitle}>{editingId ? `Edit Medicine (ID: ${editingId})` : 'Add New Medicine'}</h2>
```

Figure 12. react-app/src/App.jsx (5 of 7) – lines 190–239

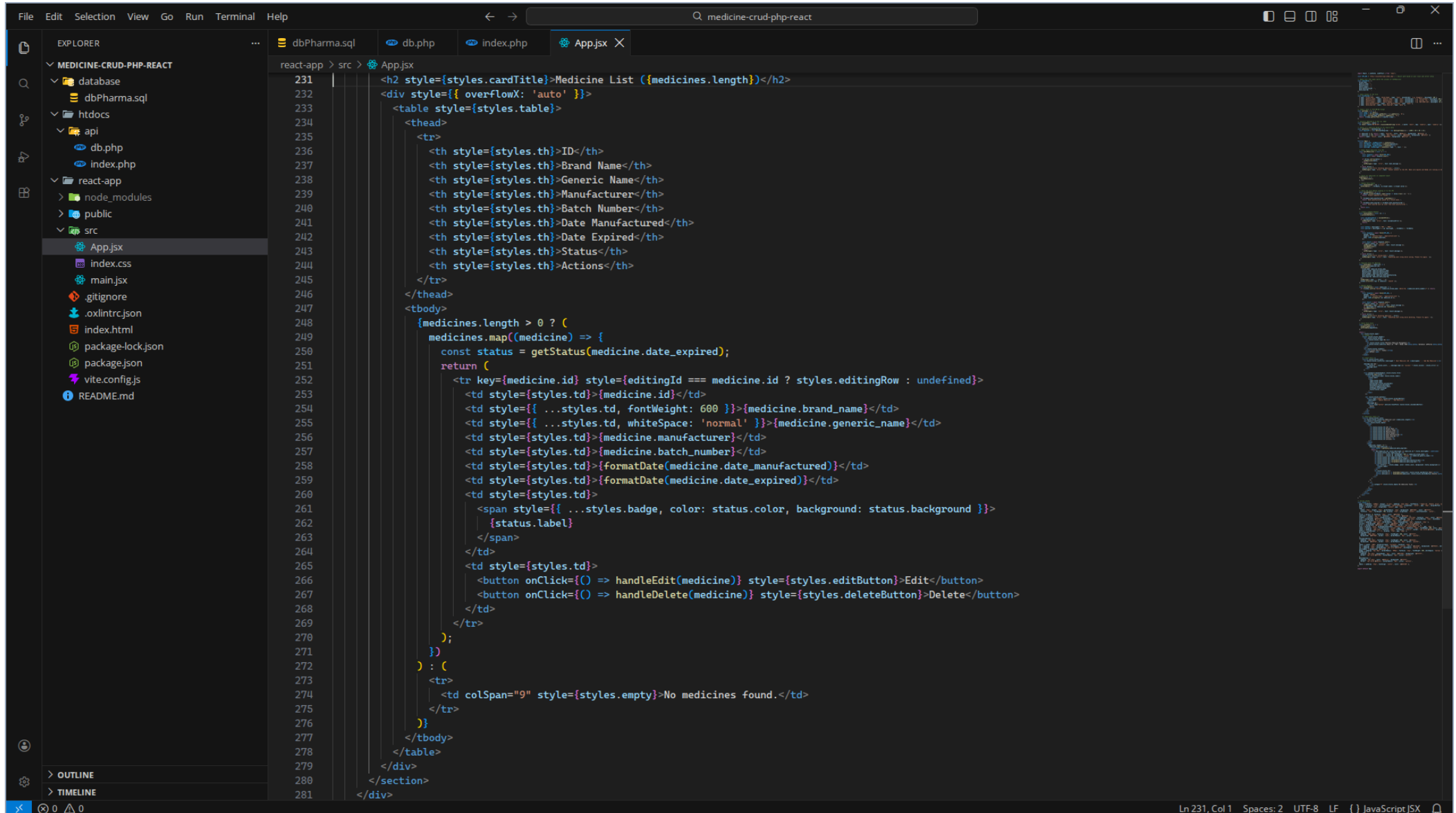
Add / Edit medicine form.



```
190  /* Input Form */
191  <section style={styles.card}>
192    <h2 style={styles.cardTitle}>{editingId ? 'Edit Medicine (ID: ${editingId})' : 'Add New Medicine'}</h2>
193
194    {message.text && (
195      <div style={{ ...styles.alert, ...(message.type === 'success' ? styles.success : styles.error) }}>
196        {message.text}
197      </div>
198    )}
199
200    <form onSubmit={handleSubmit} style={styles.form}>
201      {formFields.map((field) => (
202        <label key={field.name} style={styles.label}>
203          {field.label}
204          <input
205            type={field.type}
206            name={field.name}
207            placeholder={field.placeholder}
208            maxLength={field.maxLength}
209            value={formData[field.name]}
210            onChange={handleChange}
211            style={styles.input}
212          />
213        </label>
214      ))}
215
216      <div style={styles.buttons}>
217        <button type="submit" style={styles.primaryButton}>
218          {editingId ? 'Update Medicine' : 'Add Medicine'}
219        </button>
220        {editingId && (
221          <button type="button" onClick={resetForm} style={styles.secondaryButton}>
222            Cancel
223          </button>
224        )}
225      </div>
226    </form>
227  </section>
228
229  /* Medicines Table */
230  <section style={styles.card}>
231    <h2 style={styles.cardTitle}>Medicine List ({medicines.length})</h2>
232    <div style={{ overflowX: 'auto' }}>
233      <table style={styles.table}>
234        <thead>
235          <tr>
236            <th style={styles.th}>ID</th>
237            <th style={styles.th}>Brand Name</th>
238            <th style={styles.th}>Generic Name</th>
239            <th style={styles.th}>Manufacturer</th>
240            <th style={styles.th}>Batch Number</th>
```

Figure 13. react-app/src/App.jsx (6 of 7) – lines 231–280

Medicine list table with Edit and Delete buttons.



```
231 <h2 style={styles.cardTitle}>Medicine List ({medicines.length})</h2>
232 <div style={{ overflowX: 'auto' }}>
233   <table style={styles.table}>
234     <thead>
235       <tr>
236         <th style={styles.th}>ID</th>
237         <th style={styles.th}>Brand Name</th>
238         <th style={styles.th}>Generic Name</th>
239         <th style={styles.th}>Manufacturer</th>
240         <th style={styles.th}>Batch Number</th>
241         <th style={styles.th}>Date Manufactured</th>
242         <th style={styles.th}>Date Expired</th>
243         <th style={styles.th}>Status</th>
244         <th style={styles.th}>Actions</th>
245       </tr>
246     </thead>
247     <tbody>
248       {medicines.length > 0 ? (
249         medicines.map((medicine) => {
250           const status = getStatus(medicine.date_expired);
251           return (
252             <tr key={medicine.id} style={editingId === medicine.id ? styles.editingRow : undefined}>
253               <td style={styles.td}>{medicine.id}</td>
254               <td style={{ ...styles.td, fontWeight: 600 }}>{medicine.brand_name}</td>
255               <td style={{ ...styles.td, whiteSpace: 'normal' }}>{medicine.generic_name}</td>
256               <td style={styles.td}>{medicine.manufacturer}</td>
257               <td style={styles.td}>{medicine.batch_number}</td>
258               <td style={styles.td}>{formatDate(medicine.date_manufactured)}</td>
259               <td style={styles.td}>{formatDate(medicine.date_expired)}</td>
260               <td style={styles.td}>
261                 <span style={{ ...styles.badge, color: status.color, background: status.background }}>
262                   {status.label}
263                 </span>
264               </td>
265               <td style={styles.td}>
266                 <button onClick={() => handleEdit(medicine)} style={styles.editButton}>Edit</button>
267                 <button onClick={() => handleDelete(medicine)} style={styles.deleteButton}>Delete</button>
268               </td>
269             </tr>
270           );
271         })
272       ) : (
273         <tr>
274           <td colspan="9" style={styles.empty}>No medicines found.</td>
275         </tr>
276       )
277     </tbody>
278   </table>
279 </div>
280 </section>
281 </div>
```

Figure 14. react-app/src/App.jsx (7 of 7) – lines 281–330

Jules Aron P. Timbas

INF241

Inline styles.

```

281 | </div>
282 | );
283 | }
284 |
285 | // Inline styles
286 | const styles = {
287 |   page: { maxWidth: '1400px', margin: '0 auto', padding: '32px 24px', fontFamily: "'Segoe UI', Roboto, Arial, sans-serif", color: '#1f2937' },
288 |   header: { display: 'flex', justifyContent: 'space-between', alignItems: 'center', gap: '16px', marginBottom: '24px' },
289 |   brand: { display: 'flex', alignItems: 'center', gap: '14px' },
290 |   logo: {
291 |     width: '52px', height: '52px', borderRadius: '12px', background: '#0f766e', color: 'white',
292 |     fontSize: '22px', fontWeight: 700, display: 'flex', alignItems: 'center', justifyContent: 'center',
293 |   },
294 |   title: { margin: 0, fontSize: '26px', color: '#0f766e' },
295 |   subtitle: { margin: '4px 0 0', fontSize: '14px', color: '#6b7280' },
296 |   student: { display: 'flex', flexDirection: 'column', alignItems: 'flex-end', fontSize: '14px', color: '#374151' },
297 |   card: { background: 'white', borderRadius: '12px', padding: '20px 24px', marginBottom: '24px', boxShadow: '0 1px 3px rgba(0, 0, 0, 0.1)' },
298 |   cardTitle: { margin: '0 0 16px', fontSize: '18px', color: '#111827' },
299 |   alert: { padding: '10px 14px', borderRadius: '8px', marginBottom: '16px', fontSize: '14px' },
300 |   success: { background: '#dcfce7', color: '#166534', border: '1px solid #86efac' },
301 |   error: { background: '#fee2e2', color: '#991b1b', border: '1px solid #fca5a5' },
302 |   form: { display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(340px, 1fr))', gap: '16px' },
303 |   label: { display: 'flex', flexDirection: 'column', gap: '6px', fontSize: '13px', fontWeight: 600, color: '#374151' },
304 |   input: { padding: '10px 12px', fontSize: '14px', fontFamily: 'inherit', border: '1px solid #d1d5db', borderRadius: '8px' },
305 |   buttons: { gridColumn: '1 / -1', display: 'flex', gap: '10px' },
306 |   primaryButton: {
307 |     padding: '10px 20px', fontSize: '14px', fontWeight: 600, color: 'white',
308 |     background: '#0f766e', border: 'none', borderRadius: '8px', cursor: 'pointer',
309 |   },
310 |   secondaryButton: {
311 |     padding: '10px 20px', fontSize: '14px', fontWeight: 600, color: '#374151',
312 |     background: '#e5e7eb', border: 'none', borderRadius: '8px', cursor: 'pointer',
313 |   },
314 |   table: { width: '100%', borderCollapse: 'collapse', fontSize: '14px' },
315 |   th: { padding: '12px', textAlign: 'left', whiteSpace: 'nowrap', color: '#111827', background: '#f0f0f0', borderBottom: '2px solid #99f6e4' },
316 |   td: { padding: '12px', borderBottom: '1px solid #e5e7eb', whiteSpace: 'nowrap' },
317 |   editingRow: { background: '#fefce8' },
318 |   badge: { padding: '4px 10px', borderRadius: '999px', fontSize: '12px', fontWeight: 600, whiteSpace: 'nowrap' },
319 |   editButton: {
320 |     padding: '6px 14px', marginRight: '8px', color: '#0f766e', background: 'white',
321 |     border: '1px solid #0f766e', borderRadius: '6px', cursor: 'pointer',
322 |   },
323 |   deleteButton: {
324 |     padding: '6px 14px', color: '#b91c1c', background: 'white',
325 |     border: '1px solid #b91c1c', borderRadius: '6px', cursor: 'pointer',
326 |   },
327 |   empty: { padding: '24px', textAlign: 'center', color: '#6b7280' },
328 | };
329 |
330 | export default App;
331 |

```

Figure 15. READ – medicine list

All records of tblMedicine are loaded from the PHP API when the page opens.

Jules Aron P. Timbas
INF241

Pharmacy Medicine Mana

localhost:5173

Rx

Pharmacy Medicine Management

React JS + PHP + MySQL CRUD | Database: dbPharma | Table: tblMedicine

Jules Aron P. Timbas
INF241

Add New Medicine

Brand Name

Generic Name

Manufacturer

Batch Number

Date Manufactured

Date Expired

Add Medicine

Medicine List (5)

ID	Brand Name	Generic Name	Manufacturer	Batch Number	Date Manufactured	Date Expired	Status	Actions
5	Solmux	Carbocisteine	Unilab, Inc.	SMX-2507-087	Jul 15, 2025	Jul 14, 2027	Valid	Edit Delete
4	Lipitor	Atorvastatin Calcium	Pfizer, Inc.	LPT-2506-331	Jun 2, 2025	Jun 1, 2028	Valid	Edit Delete
3	Amoxil	Amoxicillin	GlaxoSmithKline	AMX-2308-204	Aug 21, 2023	Aug 20, 2025	Expired	Edit Delete
2	Neozep Forte	Phenylephrine + Chlorphenamine + Paracetamol	Unilab, Inc.	NZF-2411-052	Nov 5, 2024	Nov 4, 2026	Near Expiry	Edit Delete
1	Biogesic	Paracetamol	Unilab, Inc.	BGS-2503-118	Mar 12, 2025	Mar 11, 2028	Valid	Edit Delete

Figure 16. CREATE – filling in a new medicine

Brand name, generic name, manufacturer, batch number and dates of a new medicine.

Jules Aron P. Timbas
INF241

Pharmacy Medicine Mana

localhost:5173

Rx

Pharmacy Medicine Management

React JS + PHP + MySQL CRUD | Database: dbPharma | Table: tblMedicine

Jules Aron P. Timbas
INF241

Add New Medicine

Brand Name

Generic Name

Manufacturer

Batch Number

Date Manufactured

Date Expired

Add Medicine

Medicine List (5)

ID	Brand Name	Generic Name	Manufacturer	Batch Number	Date Manufactured	Date Expired	Status	Actions
5	Solmux	Carbocisteine	Unilab, Inc.	SMX-2507-087	Jul 15, 2025	Jul 14, 2027	Valid	EditDelete
4	Lipitor	Atorvastatin Calcium	Pfizer, Inc.	LPT-2506-331	Jun 2, 2025	Jun 1, 2028	Valid	EditDelete
3	Amoxil	Amoxicillin	GlaxoSmithKline	AMX-2308-204	Aug 21, 2023	Aug 20, 2025	Expired	EditDelete
2	Neozep Forte	Phenylephrine + Chlorphenamine + Paracetamol	Unilab, Inc.	NZF-2411-052	Nov 5, 2024	Nov 4, 2026	Near Expiry	EditDelete
1	Biogesic	Paracetamol	Unilab, Inc.	BGS-2503-118	Mar 12, 2025	Mar 11, 2028	Valid	EditDelete

Figure 17. CREATE – medicine added

The new medicine (ID 6) is saved and shown at the top of the list.

Jules Aron P. Timbas
INF241

Rx

Pharmacy Medicine Management

React JS + PHP + MySQL CRUD | Database: dbPharma | Table: tblMedicine

Jules Aron P. Timbas

INF241

Medicine added successfully

Brand Name

e.g. Biogesic

Generic Name

e.g. Paracetamol

Manufacturer

e.g. Unilab, Inc.

Batch Number

e.g. BGS-2503-118

Date Manufactured

mm/dd/yyyy

Date Expired

mm/dd/yyyy

Add Medicine

Medicine List (6)

ID	Brand Name	Generic Name	Manufacturer	Batch Number	Date Manufactured	Date Expired	Status	Actions
6	Ventolin	Salbutamol	GlaxoSmithKline	VTL-2601-015	Jan 20, 2026	Jan 19, 2028	Valid	Edit Delete
5	Solmux	Carbocisteine	Unilab, Inc.	SMX-2507-087	Jul 15, 2025	Jul 14, 2027	Valid	Edit Delete
4	Lipitor	Atorvastatin Calcium	Pfizer, Inc.	LPT-2506-331	Jun 2, 2025	Jun 1, 2028	Valid	Edit Delete
3	Amoxil	Amoxicillin	GlaxoSmithKline	AMX-2308-204	Aug 21, 2023	Aug 20, 2025	Expired	Edit Delete
2	Neozep Forte	Phenylephrine + Chlorphenamine + Paracetamol	Unilab, Inc.	NZF-2411-052	Nov 5, 2024	Nov 4, 2026	Near Expiry	Edit Delete
1	Biogesic	Paracetamol	Unilab, Inc.	BGS-2503-118	Mar 12, 2025	Mar 11, 2028	Valid	Edit Delete

Figure 18. UPDATE – editing a medicine

Clicking Edit loads the record into the form; the generic name is changed.

Jules Aron P. Timbas
INF241

Rx

Pharmacy Medicine Management

React JS + PHP + MySQL CRUD | Database: dbPharma | Table: tblMedicine

Jules Aron P. Timbas

INF241

Edit Medicine (ID: 6)

Brand Name

Ventolin

Generic Name

Salbutamol Sulfate

Manufacturer

GlaxoSmithKline

Batch Number

VTL-2601-015

Date Manufactured

01/20/2026

Date Expired

01/19/2028

Update Medicine

Cancel

Medicine List (6)

ID	Brand Name	Generic Name	Manufacturer	Batch Number	Date Manufactured	Date Expired	Status	Actions
6	Ventolin	Salbutamol	GlaxoSmithKline	VTL-2601-015	Jan 20, 2026	Jan 19, 2028	Valid	EditDelete
5	Solmux	Carbocisteine	Unilab, Inc.	SMX-2507-087	Jul 15, 2025	Jul 14, 2027	Valid	EditDelete
4	Lipitor	Atorvastatin Calcium	Pfizer, Inc.	LPT-2506-331	Jun 2, 2025	Jun 1, 2028	Valid	EditDelete
3	Amoxil	Amoxicillin	GlaxoSmithKline	AMX-2308-204	Aug 21, 2023	Aug 20, 2025	Expired	EditDelete
2	Neozep Forte	Phenylephrine + Chlorphenamine + Paracetamol	Unilab, Inc.	NZF-2411-052	Nov 5, 2024	Nov 4, 2026	Near Expiry	EditDelete
1	Biogesic	Paracetamol	Unilab, Inc.	BGS-2503-118	Mar 12, 2025	Mar 11, 2028	Valid	EditDelete

Figure 19. UPDATE – medicine updated

The change is saved in the database and shown in the list.

Jules Aron P. Timbas
INF241

Pharmacy Medicine Mana

localhost:5173

Rx

Pharmacy Medicine Management

React JS + PHP + MySQL CRUD | Database: dbPharma | Table: tblMedicine

Jules Aron P. Timbas
INF241

Add New Medicine

Medicine updated successfully

Brand Name

Generic Name

Manufacturer

Batch Number

Date Manufactured

Date Expired

Add Medicine

Medicine List (6)

ID	Brand Name	Generic Name	Manufacturer	Batch Number	Date Manufactured	Date Expired	Status	Actions
6	Ventolin	Salbutamol Sulfate	GlaxoSmithKline	VTL-2601-015	Jan 20, 2026	Jan 19, 2028	Valid	Edit Delete
5	Solmux	Carbocisteine	Unilab, Inc.	SMX-2507-087	Jul 15, 2025	Jul 14, 2027	Valid	Edit Delete
4	Lipitor	Atorvastatin Calcium	Pfizer, Inc.	LPT-2506-331	Jun 2, 2025	Jun 1, 2028	Valid	Edit Delete
3	Amoxil	Amoxicillin	GlaxoSmithKline	AMX-2308-204	Aug 21, 2023	Aug 20, 2025	Expired	Edit Delete
2	Neozep Forte	Phenylephrine + Chlorphenamine + Paracetamol	Unilab, Inc.	NZF-2411-052	Nov 5, 2024	Nov 4, 2026	Near Expiry	Edit Delete
1	Biogesic	Paracetamol	Unilab, Inc.	BGS-2503-118	Mar 12, 2025	Mar 11, 2028	Valid	Edit Delete

Figure 20. Validation – duplicate batch number

A batch number that already exists is rejected (UNIQUE constraint on batch_number).

Jules Aron P. Timbas
INF241

Pharmacy Medicine Mana

localhost:5173

Rx

Pharmacy Medicine Management

React JS + PHP + MySQL CRUD | Database: dbPharma | Table: tblMedicine

Jules Aron P. Timbas
INF241

Add New Medicine

Batch number already exists. Please use a different batch number.

Brand Name

Generic Name

Manufacturer

Batch Number

Date Manufactured

Date Expired

Biogesic

Paracetamol

Unilab, Inc.

BGS-2503-118

02/10/2026

02/09/2029

Add Medicine

Medicine List (6)

ID	Brand Name	Generic Name	Manufacturer	Batch Number	Date Manufactured	Date Expired	Status	Actions
6	Ventolin	Salbutamol Sulfate	GlaxoSmithKline	VTL-2601-015	Jan 20, 2026	Jan 19, 2028	Valid	EditDelete
5	Solmux	Carbocisteine	Unilab, Inc.	SMX-2507-087	Jul 15, 2025	Jul 14, 2027	Valid	EditDelete
4	Lipitor	Atorvastatin Calcium	Pfizer, Inc.	LPT-2506-331	Jun 2, 2025	Jun 1, 2028	Valid	EditDelete
3	Amoxil	Amoxicillin	GlaxoSmithKline	AMX-2308-204	Aug 21, 2023	Aug 20, 2025	Expired	EditDelete
2	Neozep Forte	Phenylephrine + Chlorphenamine + Paracetamol	Unilab, Inc.	NZF-2411-052	Nov 5, 2024	Nov 4, 2026	Near Expiry	EditDelete
1	Biogesic	Paracetamol	Unilab, Inc.	BGS-2503-118	Mar 12, 2025	Mar 11, 2028	Valid	EditDelete

Figure 21. DELETE – confirmation

The user is asked to confirm before a medicine is deleted.

Jules Aron P. Timbas
INF241

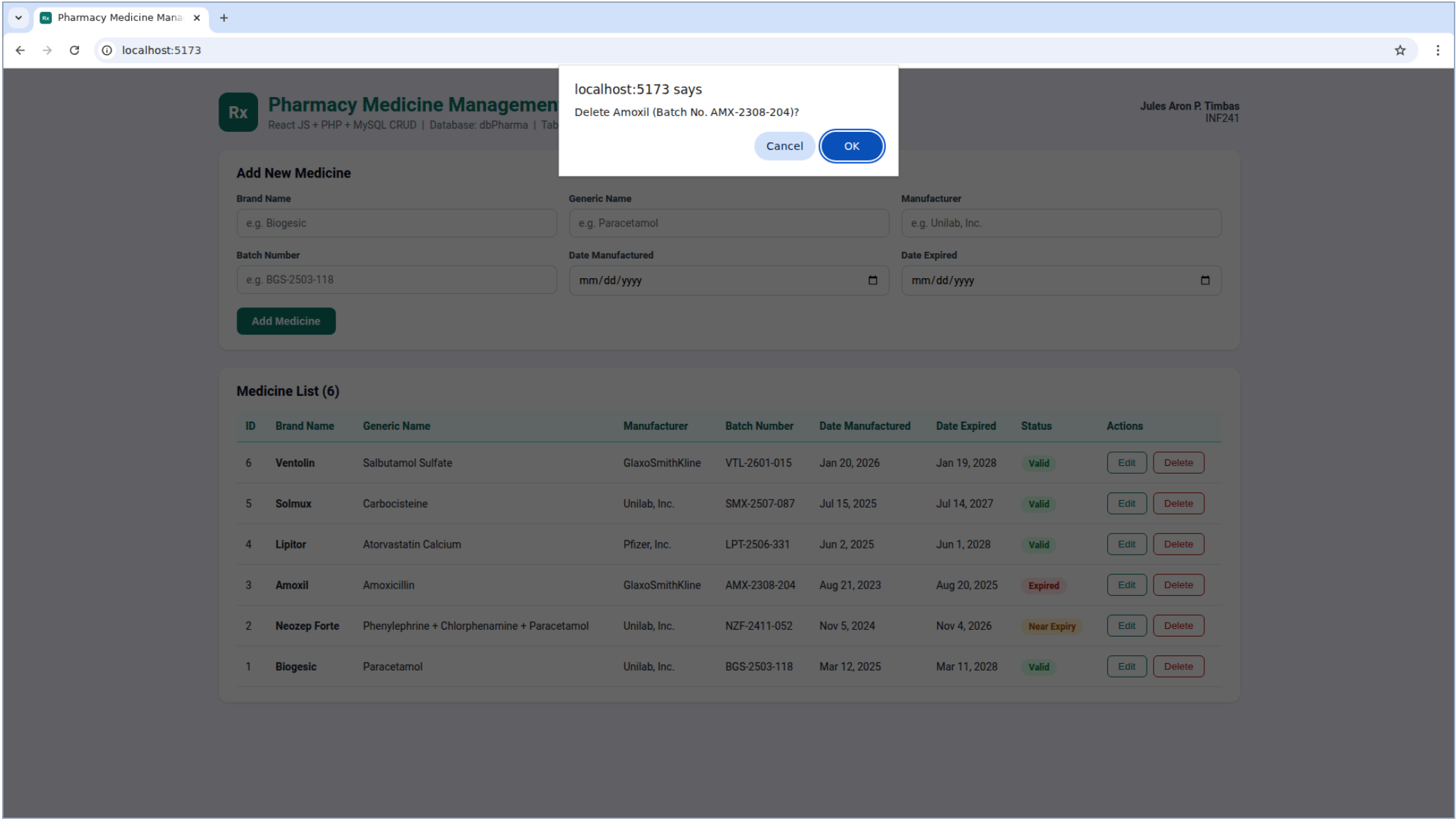


Figure 22. DELETE – medicine deleted

Amoxil (ID 3) is removed from the database and from the list.

Jules Aron P. Timbas
INF241

Pharmacy Medicine Management

React JS + PHP + MySQL CRUD | Database: dbPharma | Table: tblMedicine

Jules Aron P. Timbas
INF241

Add New Medicine

Medicine deleted successfully

Brand Name

Generic Name

Manufacturer

Batch Number

Date Manufactured

Date Expired

Add Medicine

Medicine List (5)

ID	Brand Name	Generic Name	Manufacturer	Batch Number	Date Manufactured	Date Expired	Status	Actions
6	Ventolin	Salbutamol Sulfate	GlaxoSmithKline	VTL-2601-015	Jan 20, 2026	Jan 19, 2028	Valid	EditDelete
5	Solmux	Carbocisteine	Unilab, Inc.	SMX-2507-087	Jul 15, 2025	Jul 14, 2027	Valid	EditDelete
4	Lipitor	Atorvastatin Calcium	Pfizer, Inc.	LPT-2506-331	Jun 2, 2025	Jun 1, 2028	Valid	EditDelete
2	Neozep Forte	Phenylephrine + Chlorphenamine + Paracetamol	Unilab, Inc.	NZF-2411-052	Nov 5, 2024	Nov 4, 2026	Near Expiry	EditDelete
1	Biogesic	Paracetamol	Unilab, Inc.	BGS-2503-118	Mar 12, 2025	Mar 11, 2028	Valid	EditDelete

Figure 23. tblMedicine after the CRUD operations

Jules Aron P. Timbas

INF241

SELECT * FROM tblMedicine: Ventolin was added and updated, Amoxil was deleted.

The screenshot shows a code editor with the following content:

dbPharma.sql

```

1  |=====
2  -- Activity : Medicine Management (React JS + PHP + MySQL)
3  -- Name      : Jules Aron P. Timbas
4  -- Section   : INF241
5  |=====
6
7  -- 1. Create the database
8  CREATE DATABASE IF NOT EXISTS dbPharma
9      CHARACTER SET utf8mb4
10     COLLATE utf8mb4_general_ci;
11
12  USE dbPharma;
13
14  -- 2. Create the medicine table
15  CREATE TABLE IF NOT EXISTS tblMedicine (
16      id                INT UNSIGNED NOT NULL AUTO_INCREMENT,
17      brand_name         VARCHAR(100) NOT NULL,
18      generic_name       VARCHAR(100) NOT NULL,
19      date_manufactured  DATE         NOT NULL,
20      date_expired       DATE         NOT NULL,
21      manufacturer       VARCHAR(150) NOT NULL,
22      batch_number       VARCHAR(50)  NOT NULL,
23
24      PRIMARY KEY (id),
25      CONSTRAINT uq_batch_number UNIQUE (batch_number),
26      CONSTRAINT chk_expiry_date CHECK (date_expired > date_manufactured)
27  ) ENGINE = InnoDB;
28
29  -- 3. Sample records
30  INSERT INTO tblMedicine
31  (brand_name, generic_name, date_manufactured, date_expired, manufacturer, batch_number)
32  VALUES
33  ('Biogesic', 'Paracetamol', '2025-03-12', '2028-03-11', 'Unilab, Inc.', 'BGS-2503-118'),
34  ('Neozep Forte', 'Phenylephrine + Chlorphenamine + Paracetamol', '2024-11-05', '2026-11-04', 'Unilab, Inc.', 'NZF-2411-052'),
35  ('Amoxil', 'Amoxicillin', '2023-08-21', '2025-08-20', 'GlaxoSmithKline', 'AMX-2308-204'),
36  ('Lipitor', 'Atorvastatin Calcium', '2025-06-02', '2028-06-01', 'Pfizer, Inc.', 'LPT-2506-331'),
37  ('Solmux', 'Carbocisteine', '2025-07-15', '2027-07-14', 'Unilab, Inc.', 'SMX-2507-087');

```

Terminal Output:

```

MariaDB [dbPharma]> SELECT * FROM tblMedicine;
+----+-----+-----+-----+-----+-----+-----+
| id | brand_name | generic_name | date_manufactured | date_expired | manufacturer | batch_number |
+----+-----+-----+-----+-----+-----+-----+
| 1  | Biogesic  | Paracetamol | 2025-03-12       | 2028-03-11  | Unilab, Inc. | BGS-2503-118 |
| 2  | Neozep Forte | Phenylephrine + Chlorphenamine + Paracetamol | 2024-11-05       | 2026-11-04  | Unilab, Inc. | NZF-2411-052 |
| 4  | Lipitor   | Atorvastatin Calcium | 2025-06-02       | 2028-06-01  | Pfizer, Inc. | LPT-2506-331 |
| 5  | Solmux    | Carbocisteine | 2025-07-15       | 2027-07-14  | Unilab, Inc. | SMX-2507-087 |
| 6  | Ventolin  | Salbutamol Sulfate | 2026-01-20       | 2028-01-19  | GlaxoSmithKline | VTL-2601-015 |
+----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.000 sec)

MariaDB [dbPharma]>

```